

Chris Pearce
Michael Wen
Christopher Andary
CSCI 184
Professor Tiantian Chen

Rice Strain Identification Group Project Report

1) Background

Of the many varieties of rice produced in West Africa, none has become as popular or preferred as jasmine rice. With such a high demand for this variety, countries like Ghana have experienced an increasing rate of food fraud through the substitution of jasmine with another rice variety called agra. This adulteration negatively affects consumers who are willing to spend extra money for jasmine rice, and also harms businesses that don't take part in food fraud, who get out-competed. While there are some methods of determining the exact strain of a grain of rice, they involve very expensive and time-consuming techniques in a laboratory that aren't easily accessible to developing countries.

In the Science Direct article, researchers investigated the application of pocket-sized NIR spectrometers at on-site agricultural environments to help buyers and sellers verify the quality and strain of their rice. These tools were non-invasive and were able to extract the chemical composition of rice grains for evaluation, making this a very promising solution. The goal for this project is to find another method of identifying rice grains using their images, by utilizing concepts explored and learned in class. We hope to produce similarly effective results in order to demonstrate possible real-life applications of convolutional neural networks, or CNNs.

The project idea is to create an image recognition model to detect different types of rice grains utilizing CNNs and simple linear models to identify and classify rice varieties. This idea of evaluating models for rice grain classification came from the Springer article that talks about evaluating different neural network performance on a dataset aimed to classify different grains of rice. The project aims to distinguish between visually similar rice grains and analyze how

accurately different machine learning models can perform this classification task.

Through image recognition, it can be utilized to support food quality control systems and help detect mislabeled or adulterated rice products, contributing to the prevention of food fraud. One idea is to potentially convert images to grayscale, vectorize them by flattening pixel values into a one-dimensional feature vector for use in a linear classification model, and compare its performance to a CNN that leverages spatial relationships to learn features such as grain boundaries, texture, and shape.

2) Dataset

In this project, we will be utilizing a rice image dataset provided by Kaggle. There are originally 75,000 examples in the dataset, which we split into a training dataset of 60,000 images and a validation dataset of 15,000 images. There are a total of 5 classes in this dataset, consisting of arborio, basmati, ipsala, jasmine, and karacadag rice strains. Based on the image below, you can see how different strains of rice are typically shaped uniquely, yet rice grains of the same strain retain roughly the same figure.

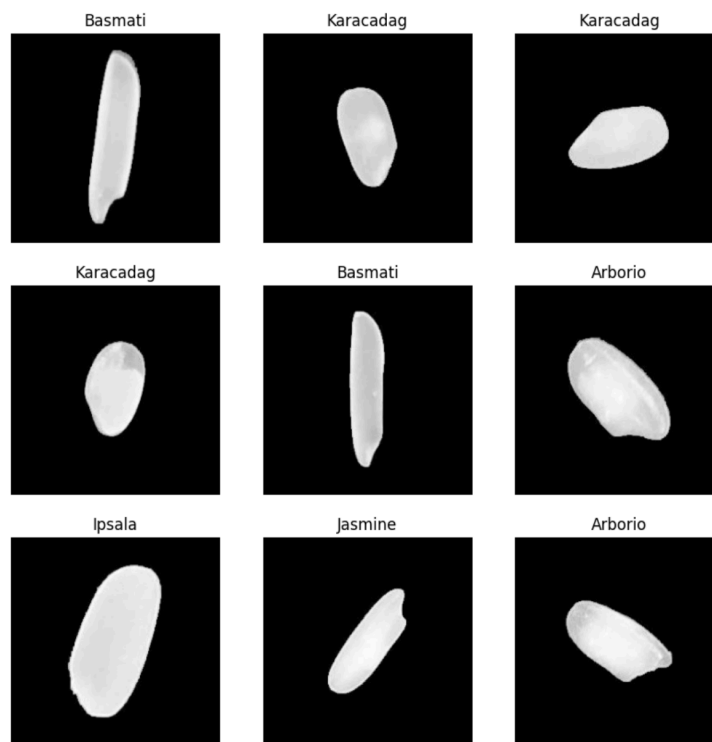


Figure 1: Examples of training images

Despite these differences, it is still possible to mix similar looking grains within barrels or bags of rice, such as basmati rice with jasmine rice to artificially increase the weight of the product while still charging a higher price of jasmine rice. This is why it's important to develop a quick and reliable method of discerning different strains of rice which would otherwise be impossible to the human eye at such volumes.

3) Image Classification

As previously mentioned, we intend to develop a convolutional neural network image classification system, and to explore how this particular algorithm is able to perform better compared to simpler linear neural network models. [This article](#) explains that an artificial neural network (ANN), particularly the linear version of this that we use in our project, is a network of interconnected neurons in which data flows forward from input to output. This works well with tabular or structured data, being able to learn complex patterns and relationships between different features in a dataset. With an adjustable number and size of hidden layers, this model provides a simple structure that is relatively fault-tolerant and easy to set up. However, since the number of learnable parameters increases sharply with layer count and size, this algorithm quickly becomes hardware-dependent, and becomes a bottleneck in scenarios with large amounts of features. This symptom is exacerbated when handling large amounts of images, where relationships between individual pixels become too complicated and abstract for ANNs to create meaningful classifications.

In this project which relies heavily on image classification, convolutional neural networks (CNN) excel at recognizing patterns at a higher level. It is designed to process grid-like data such as images in videos, and uses convolutional and pooling layers to take in raw data and automatically extracts higher level meaning from features while reducing dimensionality. CNNs do this by utilizing a shifting filter or kernel of weight parameters throughout an image, performing cross correlation on each section. The resultant is typically a smaller matrix, containing richer and more valuable data. In each layer, this step is followed by a pooling layer that reduces dimensionality further while retaining the most essential information. After an image has passed through all

convolution and pooling layers, the data is then flattened into a one dimensional vector and passed through a final fully connected neural network, which then outputs a class prediction.

At a higher level, this means that CNNs can detect more abstract relationships such as edges and shapes without need for manual engineering, and utilizes pooling and weight sharing to reduce learnable parameters. This would require less resources than a typical NN, while dramatically reducing the amount of time it takes to converge. As a result, this algorithm is known for its efficient performance and high accuracy for image recognition tasks. One tradeoff, however, is its requirement for large amounts of training data before it becomes effective— a problem we won't be facing given the volume of our dataset.

4) Experimental Design (CNN)

In order to evaluate the effectiveness of the convolutional neural network for the grain classification, we implemented a CNN and trained it on the rice image dataset described previously. The goal of this experiment is to determine how well a CNN can learn visual features from rice grain images and accurately classify them into the five rice types: arborio, basmati, ipsala, jasmine, and karacadag. Before training the model, the dataset was prepared by resizing all images to 224 x 224 pixels and converting them to grayscale. The dataset originally contains 75,000 images, which were split into 60,000 training images and 15,000 validation images. During training we used a batch size of 64, which results in approximately 938 batches per epoch when processing the 60,000 training images. A rescaling layer was also applied so that pixel values fall between 0 and 1 which helps stabilize the training process.

The CNN architecture used in this experiment consists of several convolutional and pooling layers designed to progressively extract higher level features from the images. The first convolution layer contains 16 filters with a kernel size of 4x4 and uses a ReLU activation function. This layer begins learning the basic visual features such as edges and grain boundaries. A max pooling layer is then applied to reduce the spatial dimensions of the feature maps while preserving the most important information about the image. The second convolutional layer increases the number of filters to 32, which again uses a 4x4

kernel and ReLU activation. This allows the network to capture more detailed information, such as grain shape and texture. Another max pooling layer is applied to further reduce dimensionality. A third convolution layer with 64 filters is then used to capture more complex features that help distinguish between the different rice types. After the convolution and pooling layers, the feature maps are flattened into a one-dimensional vector and passed to a fully connected output layer that performs the final classification. The model was trained using Stochastic Gradient Descent as the optimizer and sparse Categorical Cross-entropy (used for multi class classification) along with the softmax activation function output layer for 10 training epochs.

The softmax function converts the raw output values of the network into probabilities for each class. Mathematically, softmax takes a vector of scores and transforms each value using an exponential function, then normalizes them by dividing by the sum of all exponentials. This ensures that the output values are between 0 and 1 and that they sum to 1, which allows the model to interpret its outputs as probabilities for each rice class. The predicted class is then chosen as the one with the highest probability. To evaluate the model's performance, we tracked both training accuracy and validation accuracy, as well as training loss and validation loss.

5) CNN Training Results

The results of training the CNN model are shown in the accuracy and loss graphs. The training accuracy increases steadily across the epochs starting slightly above 90% in the first epoch and reaching approximately 98% accuracy by the final epoch. This indicates that the model is successfully learning important patterns from the rice grain images. The validation accuracy remains consistently high which is generally between 97% and 98% and closely follows the training accuracy. Because the validation accuracy remains similar to the training accuracy, this suggests that the model generalizes well to unseen data rather than simply memorizing the training examples. The loss curves show a similar pattern, which shows the training loss decreases consistently as training progresses. This indicates that the model's predictions are becoming more accurate. The validation loss also decreases overall although there are fluctuations between some epochs. These fluctuations are common when training neural networks and typically occur due to differences in the validation

batches.

Overall, the CNN model achieved high classification accuracy and maintained relatively stable validation performance, suggesting that it is effective at learning the visual differences between the different grains provided. Overall, the CNN achieved approximately 98% training accuracy and around 97 – 98% validation accuracy after 10 epochs. This demonstrates strong performance on the rice grain classification task.

5) Linear Experimental Design and Training Results

For the linear neural network, we once again scaled our images to a 224x224 scale, changing them to grayscale intensity scalars. In order to make the most direct comparisons with CNN as possible, we retained most of the same hyperparameters used. This means that our batch size was 64, using 10 epochs which created 938 batches per epoch for the training dataset. The neural network layers consist of a 50,176x1 input layer, feeding into hidden layers of sizes 256, then 128. The final output layer leaves us with a 5x1 vector containing a class prediction.

Since this model is used to determine the efficacy of our CNN, we decided to leave out activation functions in order to retain linearity. As a result, we expect this model to perform worse, as it is unable to account for more complicated patterns detectable by introducing activation functions like ReLU, sigmoid, or hyperbolic tangent. This means that our linear model will be much more sensitive to orientation of the rice, relying on manual fine tuning when it comes to capturing images of rice grains. For example, when making a prediction on a rice grain that isn't centered well or oriented in a strange way, we can expect the linear model to be inaccurate.

However, since our rice image dataset is very clean (all the rice grains are clearly visible and centered well), we were able to produce very accurate results. The model had pretty much converged within the first epoch, and the validation accuracy hovered around 95–97% afterward. This value may seem very close to our CNN model, but we must keep in mind that this data is very clean, and will likely perform worse in more practical, real life settings. Next, we plan to upload

our own images and utilize image detection to evaluate our models with real world examples.

6) Comparison of the Two Models

Both the convolutional neural network and the linear neural network were trained and evaluated on the same rice image dataset in order to compare their performance on the classification task at hand. While both models were able to achieve high accuracy there were some noticeable differences in their performance and how they processed the image data.

The CNN achieved approximately 97 - 98% validation accuracy, while the linear neural network achieved around 95% validation accuracy on the dataset. Although the difference in accuracy is relatively small, the CNN consistently performed slightly better across training the validation epochs. This improvement is largely due to the CNNs ability to preserve spatial relationships within the image through convolutional filters allowing it to detect features such as edges, grain boundaries and the texture. In contrast, the linear model first flattens each image into a one dimensional vector before classification. This process removes the spatial structure of the image and treats each pixel as an independent feature. As a result, the model may struggle to capture more complex visual patterns that help distinguish between similar rice grain shapes.

Another important difference between the two models is their scalability. CNNs are specifically designed for image data and use convolutional filters and pooling operations to reduce dimensionality while preserving important visual information. This makes them more efficient for larger image datasets and more complex classification tasks. Additionally, CNNs can be more computationally efficient during training for image data because convolutional layers reuse weights and reduce the number of parameters compared to fully connected layers on flattened images. Linear models, while simpler and faster to implement, generally required manual feature engineering or very clean datasets to achieve similar performance.

Overall, while both models performed well on this dataset, the CNN demonstrated stronger performance and greater capability in learning visual features from the rice grain images.

7) Extra Credit: Image Detection

Looking at our training data, our group wanted to create an Image detection model utilizing our CNN as a classifier. In class, we learned about a couple of different ways to handle object detection; however, after doing extra research, I learned that Alexnet and the other methods may not handle our specific case due to the two main issues:

- 1) Our data set is trained over single grains of rice
- 2) Our data is white pixels with a black background. (essentially grey-scaled)

Instead of modifying the classifier itself to handle the boundary boxes, our solution was to utilize contour and masking to create threshold values to size our boxes.

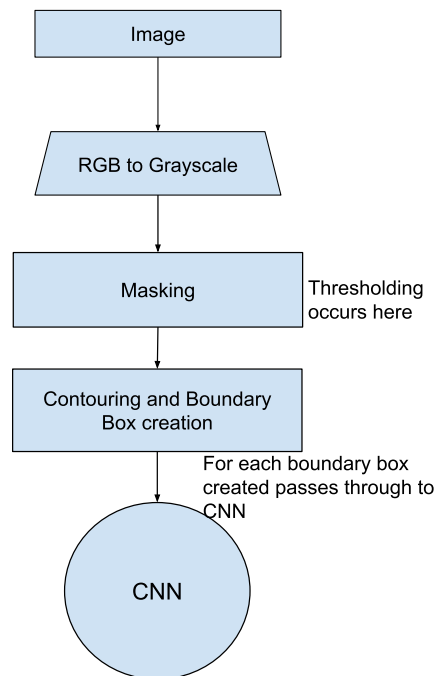


Figure 2: Image detection pipeline

I will introduce each technique one by one. The first step we needed to do was to grey-scale the images to do this we utilized OpenCV RGB to Greyscale conversion tool. Following this step, we need to create a masking layer that creates a threshold of the intensity value of the Pixel and creates a function similar to Relu, where values above T =threshold will be equal to 1 while values

below the threshold will be equal to 0 ([OpenCV thresholding](#)). After we are able to identify the pixels that are active/rice we need to generate the boxes themselves. To do this, we utilized the OpenCV Contour and boundary box functions, respectively. The contour function utilizes graph algorithms and connected-component analysis to detect continuous boundaries of objects within a binary image, and it returns us width and height values in a contour object([OpenCV contour](#)). We then iterate through these objects and utilize the OpenCV boundary box function to apply it to our grayscale image to extract the image within the box and rescale it to pass it through our Neural network. To modify threshold values to see how the model is detecting the boxes themselves, I applied the threshold boxes and pasted them on the original image.

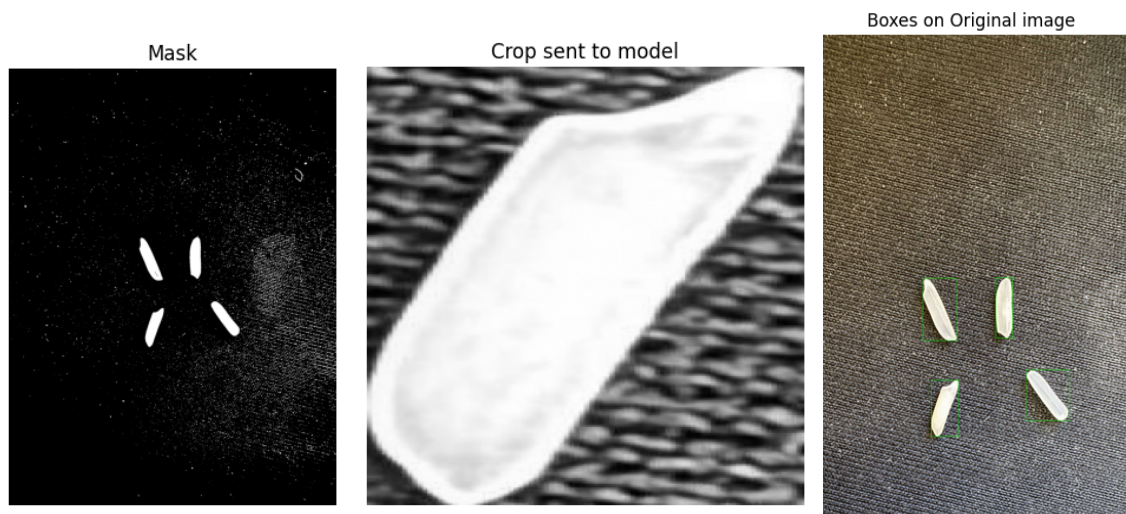


Figure 3: Examples of masking, a resized image that was sent to CNN, and the original image with a green bounding box. (Left to Right)

Analysis

Due to how scaling and padding work, our images had trouble detecting what type of rice it was sometimes, but were close when identifying the difference between long and short grain rice, i.e., it may detect Basmati rice as Jasmine. Another issue was background and threshold values, due to the fact that the threshold was hard-coded by Michael and chosen through trial and error. Depending on how bright an image is, the accuracy of the detection count and boundary box creation may vary drastically.

References

Teye, Ernest, and Charles L. Y. Amuah. "Rice Varietal Integrity and Adulteration Fraud Detection by Chemometrical Analysis of Pocket-Sized NIR Spectra Data." *Applied Food Research*, vol. 2, no. 2, Dec. 2022, article 100218,

<https://doi.org/10.1016/j.afres.2022.100218>

Shuvo, Syme Al-Azad, et al. "Image-Based Classification of Bangladeshi Rice Varieties Using Deep Convolutional Neural Networks." *Agricultural Products Processing and Storage*, vol. 2, article 26, 19 Jan. 2026,

<https://doi.org/10.1007/s44462-026-00061-9>

GeeksforGeeks. "Difference between ANN, CNN and RNN." *GeeksforGeeks*,

<https://www.geeksforgeeks.org/deep-learning/difference-between-ann-cnn-and-rnn/>

TensorFlow. "Convolutional Neural Network (CNN) Tutorial." *TensorFlow Documentation*, <https://www.tensorflow.org/tutorials/images/cnn>

TensorFlow. "Keras Sequential Model Guide." *TensorFlow Documentation*, https://www.tensorflow.org/guide/keras/sequential_model

OpenCV. "Image Thresholding." *OpenCV Documentation*, https://docs.opencv.org/4.x/d7/d4d/tutorial_py_thresholding.html

YouTube. "Convolutional Neural Networks Explained." *YouTube*, <https://www.youtube.com/watch?v=LMamYJYnws8>

YouTube. "How CNNs Work." *YouTube*, <https://www.youtube.com/watch?v=lBMosWq8NKO>

<https://www.geeksforgeeks.org/deep-learning/difference-between-ann-cnn-and-rnn/>

<https://www.tensorflow.org/tutorials/images/cnn> documentation

[https://docs.opencv.org/4.x/d7/d4d/tutorial_py_thresholding.html?](https://docs.opencv.org/4.x/d7/d4d/tutorial_py_thresholding.html)

https://docs.opencv.org/3.4/d7/d4d/tutorial_py_thresholding.html

<https://www.youtube.com/watch?v=LMamYJYnws8>

<https://www.youtube.com/watch?v=lBMosWq8NKO>

https://www.tensorflow.org/guide/keras/sequential_model